

```

    val pool = Executors.fixedThreadPool(8)
    unsafePerformIO(pureMain(args))(pool)
  }

  def pureMain(args: IndexedSeq[String]): IO[Unit]
}

```

Our actual program goes here, as an implementation of `pureMain` in a subclass of `App`. It also takes the arguments as an immutable `IndexedSeq` rather than a mutable `Array`.

We want to make a distinction here between effects and *side* effects. The `pureMain` program itself isn't going to have any side effects. It should be a referentially transparent expression of type `IO[Unit]`. The performing of effects is entirely contained within `main`, which is *outside the universe of our actual program*, `pureMain`. Since our program can't observe these effects occurring, but they nevertheless occur, we say that our program has effects but not side effects.

13.7 Why the IO type is insufficient for streaming I/O

Despite the flexibility of the `IO` monad and the advantage of having I/O actions as first-class values, the `IO` type fundamentally provides us with the same level of abstraction as ordinary imperative programming. This means that writing efficient, streaming I/O will generally involve monolithic loops.

Let's look at an example. Suppose we wanted to write a program to convert a file, `fahrenheit.txt`, containing a sequence of temperatures in degrees Fahrenheit, separated by line breaks, to a new file, `celsius.txt`, containing the same temperatures in degrees Celsius. An algebra for this might look something like this:¹⁸

```

trait Files[A]
case class ReadLines(file: String) extends Files[List[String]]
case class WriteLines(file: String, lines: List[String])
  extends Files[Unit]

```

Using this as our `F` type in `Free[F,A]`, we might try to write the program we want in the following way:

```

val p: Free[Files,Unit] = for {
  lines <- Suspend { (ReadLines("fahrenheit.txt")) }
  cs = lines.map(s => fahrenheitToCelsius(s.toDouble).toString)
  _ <- Suspend { WriteLines("celsius.txt", cs) }
} yield ()

```

This works, although it requires loading the contents of `fahrenheit.txt` entirely into memory to work on it, which could be problematic if the file is very large. We'd prefer to perform this task using roughly constant memory—read a line or a fixed-size buffer full of lines from `fahrenheit.txt`, convert to Celsius, dump to `celsius.txt`, and repeat. To achieve this efficiency we could expose a lower-level file API that gives access to I/O handles:

¹⁸ We're ignoring exception handling in this API.

```

trait Files[A]
case class OpenRead(file: String) extends Files[HandlerR]
case class OpenWrite(file: String) extends Files[HandlerW]
case class ReadLine(h: HandlerR) extends Files[Option[String]]
case class WriteLine(h: HandlerW, line: String) extends Files[Unit]

trait HandlerR
trait HandlerW

```

The only problem is that we now need to write a monolithic loop:

```

def loop(f: HandlerR, c: HandlerW): Free[Files, Unit] = for {
  line <- Suspend { ReadLine(f) }
  _ <- line match {
    case None => IO.unit()
    case Some(s) => Suspend {
      WriteLine(fahrenheitToCelsius(s.toDouble))
    } flatMap (_ => loop(f, c))
  }
} yield b

def convertFiles = for {
  f <- Suspend(OpenRead("fahrenheit.txt"))
  c <- Suspend(OpenWrite("celsius.txt"))
  _ <- loop(f, c)
} yield ()

```

There's nothing inherently wrong with writing a monolithic loop like this, but it's not composable. Suppose we decide later that we'd like to compute a five-element moving average of the temperatures. Modifying our loop function to do this would be somewhat painful. Compare that to the equivalent change we might make to `List`-based code, where we could define a `movingAvg` function and just stick it before or after our conversion to Celsius:

```

def movingAvg(n: Int)(l: List[Double]): List[Double]

cs = movingAvg(5)(lines.map(s => fahrenheitToCelsius(s.toDouble))).
  map(_.toString)

```

Even `movingAvg` could be composed from smaller pieces—we could build it using a generic combinator, `windowed`:

```

def windowed[A](n: Int, l: List[A])(f: A => B)(m: Monoid[B]): List[B]

```

The point is that programming with a composable abstraction like `List` is much nicer than programming directly with the primitive I/O operations. Lists aren't really special in this regard—they're just one instance of a composable API that's pleasant to use. And we shouldn't have to give up all the nice compositionality that we've come to expect from FP just to write programs that make use of efficient, streaming I/O.¹⁹ Luckily we don't have to. As we'll see in chapter 15, we get to build whatever abstractions we

¹⁹ One might ask if we could just have various `Files` operations return the `Stream` type we defined in chapter 5. This is called *lazy I/O*, and it's problematic for several reasons we'll discuss in chapter 15.